

CSS 书写顺序: (1) 定位、浮动、display (2) 盒子模型 = margin、border、padding、宽高、背景色 (3) 文字

HTML 结构化语言 结构

CSS 解释性语言 表现 → “网页”

JavaScript 脚本语言 行为

CSS (Cascading Style Sheet) 是一种用来装饰 HTML 的标记集合, 是对 HTML 标记的扩展, 可以进一步美化页面。

CSS 按位置可分为 3 种:

① 内嵌样式

直接将样式写入元素的 style 属性

例如 `<p style="font-size: 20px; color: red;"> 文字 </p>`

② 内部样式表

例如 `<style>
 h1 {
 text-align: center;
 color: green;
 }
</style>`

③ 外部样式表

例如

CSS/ lesson 2. CSS:

```
p.content {  
  width: 200px;  
  height: 800px;  
  background-color: blue;  
  text-align: center;  
  line-height: 200px;  
}
```

```
#pink {  
  color: pink;  
}
```

默认情况下, 块级元素的宽度会自动撑满父元素整行。父元素是<body>就撑满浏览器宽度, 高度由内容决定

```
.school, .college { // 并集选择器  
  color: purple;  
}
```

```
lesson2.html:  
<!DOCTYPE html>  
<html lang="zh-CN">  
  <head>  
    <meta charset="UTF-8">  
    <meta name="viewport"  
      content="width=device-width, initial-scale=1.0">  
    <title> 第2节课 </title>  
    <link rel="stylesheet" href="css/lesson2.css">  
  <body>  
    <p class="content"> 类选择器 </p>  
    <p id="pink"> id选择器 </p>  
    <h1 class="school"> 安徽工业大学 </h1>  
    <p class="college"> 计算机科学与技术学院 </p>  
  </body>  
</html>
```

CSS 注释: /* */

样式选择器：这个样式适合于哪些元素

① 标签选择器

对指定的标签采用统一的样式

```
h1 { color: red; font-size: 14px; }
```

选择器 属性 值

② 类(class)选择器

```
.important { color: red; } ... <h1 class="important">...</h1>
```

一个标签可以有多个class, 用空格分割
用于可以重用的标记

```
<p class="class1 class2">文字</p> ✓
```

```
<p id="id1 id2">文字</p> X
```

③ id 选择器

为指定id的元素设定样式

```
#green { color: green; } ... <p id="green">...</p> ...
```

用于标识页面唯一的区域(用于脚本编程)

注意 id 和 class 的命名规范：

① 具备语义性的单词, 不能用数字/拼音/符号

② 用“-”连接多单词, 不能直接单词拼接/驼峰命名, 但连接层级≤4层

③ 适当缩写

④ 交集选择器 (标签 + class) “.”

使同样的样式名对不同的标签有不同的样式

```
h1.important { color: red; } <h1 class="important">...</h1>  
p.important { color: green; } <p class="important">...</p>
```

⑤ 并集选择器 “,”

同时选择多个标签、class 等

```
.important, .content { color: red; }
```

```
<h1 class="important">...</h1>  
<p class="content">...</p>
```

后代选择器

匹配属于指定元素后代的所有元素

```
div p { /* 选择<div>元素内所有<p>元素 */  
    background-color: yellow;  
}
```

子选择器 (只能选择亲儿子, 而其它后代例如孙子都不行)

```
div > p {  
    background-color: yellow;  
}
```

序选择器

- :first-child 选取属于其父元素的第一个子元素
- :last-child 选取属于其父元素的最后一个子元素
- :nth-child(n) 选取属于其父元素的第n个子元素
- :nth-last-child(n) 从最后一个开始计数, 选取属于其父元素的第n个子元素
- :not(选择器) 例如 li:not(.box) 选取li元素, 排除 class为"box"的li元素

伪类选择器

伪类定义元素的特殊状态, 伪类选择器设置元素处于某种特殊状态的样式

按 顺 序 ↓	:link	超链接未被访问时的状态
	:visited	超链接已被访问过的状态
	:hover	鼠标悬停时的状态
	:active	激活状态 (一般指鼠标点击)
	:focus	选取获得焦点的元素

通配符选择器

是所有选择器中作用范围最广的, 能匹配页面中所有元素

一般用于统一默认的 margin 和 padding

```
* {  
    margin: 0;  
    padding: 0;  
}
```

例.

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>例子</title>
  <style>
```

```
    /* {
      通配符 选择器 {
        margin: 0;
        padding: 0;
      }
      body {
        padding: 20px;
      }
      .container {
        border: 1px solid #333;
        padding: 10px;
        margin-bottom: 20px;
      }
      .box {
        border: 1px dashed #333;
        padding: 10px;
        margin: 10px 0;
      }
      后代选择器 div p {
        background-color: yellow;
      }
```

```
      子选择器 .container > p {
        color: red;
      }
```

```
      序选择器 .list: first-child, .list: last-child {
        font-weight: bold;
      }
```

```
      并集选择器 .list: nth-child(2), .list: nth-last-child(2) {
        text-decoration: underline;
        font-style: italic;
      }
```

```
      后代选择器, 序选择器 .list li: not(.skip) {
        margin: 30px 0;
      }
```

```
      a: link {
        color: blue;
      }
      a: visited {
        color: purple;
      }
      a: hover {
        color: orange;
        text-decoration: none;
      }
      a: active {
        color: red;
      }
      .list-container {
        margin: 20px 0;
      }
```

```
</body>
```

```
<div class="container">
  <p> 黄色背景, 红色字 </p>
  <div class="box">
    <p> 黄色背景, 黑色字 </p>
  </div>
</div>
```

```
<ul class="list">
  <li> 加粗 </li>
  <li> 下划线, 斜体 </li>
  <li class="skip"> 被排除 </li>
  <li> 下划线, 斜体 </li>
  <li> 加粗 </li>
</ul>
```

```
<div class="link-container">
  <a href="https://www.baidu.com">
    点前蓝色, 悬停时橘色, 点时红色, 点后紫色
  </a>
</div>
```

```
<div class="form-group">
  <label> 姓名: </label>
  <input type="text">
</div>
```

```
</body>
```

```
</html>
```

```
      输入框获得焦点
      input: focus {
        border: 2px solid green;
        outline: none;
      }
```

注意 margin 的垂直外边距折叠规则!

处于普通文档流(无 float, 无 position: absolute/fixed, 无 display: flex/grid) 的相邻块级兄弟元素的垂直方向外边距会出现该现象

(均正/负则取绝对值更大者, 反之取二者之和)

要求“无阻挡”, 即二者之间没有 border、padding、行内、overflow: hidden 父设置

字体

font-size → 大小

font-style → normal, italic, oblique (后面2个: 斜体)

font-weight → normal, bold, 100~900间的整百数

color → 颜色

边框

border: 1px solid/dashed/dotted/none blue;

亦可写作
border-width: 1px;
border-style: dotted;
border-color: blue;

显示模式

标签(标记或元素)的显示方式

① 块级元素 (block)

例如 div, p, h1~h6, ul, li, form, header, footer...

特点: 独占一行, 宽度默认为父元素宽度而高度由自身内容撑开, 可以设置宽、高

② 行内元素 (inline)

例如 a, span, b, u, strong ...
加粗 (有语义)
下划线

特点: 不独占一行, 宽度、高度默认由内容撑开, 不可设置宽、高

③ 行内块元素 (inline-block)

例如 input, textarea, button, select ...

特点: 不独占一行, 宽度、高度默认由内容撑开, 可以设置宽、高

注: image 标签具有行内块元素的特点, 但事实上它在测试工具中的结果是 inline

为了改变元素的默认显示特点, 使之符合布局要求, 可进行显示模式转换:

display: block/inline/inline-block;

CSS 的继承性:

子元素默认继承父元素的样式.

基本上文字控制属性都可以继承, 例如 color, font-weight, font-size, font-family, line-height...

盒子模型

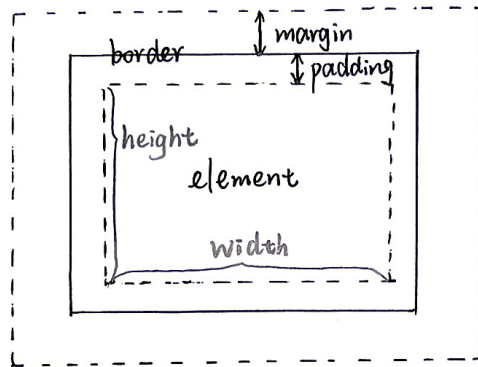
CSS规定每个盒子分别由内容区域(content), 内边距区域(padding), 边框区域(border), 外边距区域(margin)构成
元素的宽和高包含什么由 box-sizing 决定:

box-sizing: content-box (默认) 仅包含内容区域而不包含 padding, border, margin

box-sizing: border-box 包含内容区域, padding, border 而不包含 margin

CSS 盒子模型的内减模式, 好处: 加了 border, padding 无需做减法, 不用手动修改 width, height 的值

默认情况下:



页面中的每个标签都可看作一个盒子, 可方便页面布局

padding 不允许负值 top → right → bottom → left

属性值: length 以px, pt, cm等单位

% 以包含元素宽度的百分比计

inherit 从父元素继承内边距

margin 允许负值 top → right → bottom → left

属性值: auto 浏览器计算

length 以px, pt, cm等单位

% 以包含元素宽度的百分比计

inherit 从父元素继承内边距

padding 与之类似

margin: 100px 99px 100px
↑ ↑ ↑
上 左、右 下

margin: 100px 99px
↑ ↑
上、下 左、右

border 不允许负值 top → right → bottom → left

① border-width: 1px 2px 3px 4px;
border-style: solid dashed dotted none;
border-color: red orange yellow green;

② border-bottom: 1px solid blue;

③ border-bottom-width: 1px;
border-bottom-style: solid;
border-bottom-color: blue;

例. 导航条 (初级版)

加悬停效果, 鼠标停到某个导航, 该导航背景色从红色变为橙色



```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title> 初级版</title>
  <style>
    .nav-item {
      display: inline-block;
      width: 100px;
      height: 50px;
      background-color: red;
      margin: 5px;
      color: white;
      text-align: center;
      line-height: 50px;
      cursor: pointer; // 鼠标悬停时显示"手形"
    }
    .nav-item :hover {
      background-color: orange;
    }
  </style>
</head>
```

```
<body>
  <div class="nav-container">
    <div class="nav-item">导航1</div>
    <div class="nav-item">导航2</div>
    <div class="nav-item">导航3</div>
    <div class="nav-item">导航4</div>
    <div class="nav-item">导航5</div>
  </div>
</body>
</html>
```

要想实现这个整体在页面中^{垂直}水平居中, 则还需添加:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}

.nav-container {
  width: 1000px;
  padding: 20px;
  display: flex;
  justify-content: center;
  align-items: center;
}
```

例. 新闻列表

外面的盒子:

宽 500px, 高 400px, 边框 1px, 实线,
颜色 #ccc, 要求: 版心居中, 距页面上边缘 50px, 内边距 42px 30px 0

最新文章

小米的39亿, 到底赚了

GPU的历史性时刻

小米造车, 现金怎么融“钱” 融资?

美团优选“侠”突围了?

华为麒麟芯片, 王者归来!

里面的盒子:

标题: 下边框 1px, 实线, 颜色 #ccc,
字号 30px, 行高 30px, padding-bottom: 9px,
链接内容: 高 50px, 下边框 1px, 虚线,
颜色 #ccc, 文本缩进 28px, 纵向居中,
字号 18px, 字体颜色 #666

CSS/新闻列表.css:

```
*{
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
.news {
  width: 500px;
  height: 400px;
  border: 1px solid #ccc;
  margin: 50px auto 0;
  padding: 42px 30px 0;
}
.news h2 {
  border-bottom: 1px solid #ccc;
  font-size: 30px;
  line-height: 30px; 或 1 或 1em
  padding-bottom: 9px;
}
.news ul {
  list-style: none;  // 去小黑点
}
.news ul li {
  height: 50px;
  border-bottom: 1px dashed #ccc;
  padding-left: 28px;  // 缩进 28px
}
.news ul li a {
  line-height: 50px;
  font-size: 18px;
  color: #666;
  text-decoration: none;  // 去下划线
  display: inline-block;
```

新闻列表.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width,
    initial-scale=1.0">
  <title>新闻列表</title>
  <link rel="stylesheet" href="css/新闻列表.css">
</head>
<body>
  <div class="news">
    <h2>最新文章</h2>
    <ul>
      <li><a href="#">...</a></li>
      <li><a href="#">...</a></li>
      ...
      <li><a href="#">...</a></li>
    </ul>
  </div>
</body>
</html>
```

例. 导航条(升级版)

新浪导航

新浪导航

新浪导航

新浪导航

布局顺序: 从外到内, 从上到下

每个盒子的样式: 宽高、背景色、border、padding、margin、color、text-、font- ...

外面的盒子: 高40px, 上边框3px, 实线, 颜色#ff8500, 下边框1px, 实线, 颜色#edeef0

里面的盒子: 宽80px, 高40px, 悬停颜色#edeef0, 字体12px, 颜色#4c4c4c

css/lesson3.css:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
.container {
  height: 40px;
  border-top: 3px solid #ff8500;
  border-bottom: 1px solid #edeef0;
}
.container a {
  display: inline-block;
  text-decoration: none;
  width: 80px;
  height: 40px;
  font-size: 12px;
  color: #4c4c4c;
  text-align: center;
  line-height: 40px;
}
.container a: hover {
  background-color: #edeef0;
}
```

lesson3.html:

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title> 第三节课</title>
  <link rel="stylesheet" href="css/lesson3.css">
</head>
<body>
  <div class="container">
    <a href=""> 新浪导航</a>
    <a href=""> 新浪导航</a>
    <a href=""> 新浪导航</a>
    <a href=""> 新浪导航</a>
  </div>
</body>
</html>
```

优化版. 如何使导航栏可以适应不同长度的字符串?

删除宽度设置, 并且设置左右padding为16px 即 padding: 0 16px; 更美观

float 浮动作用 (现在逐渐被淘汰, 不推荐)

作用1: 图文环绕

例如

```

:
:
:
.article {
  width: 100px;
  margin: 30px auto;
  padding: 20px;
  line-height: 1.5;
}
.article-img {
  float: left;
  width: 200px;
  margin: 0 20px 20px 0;
}
:
:
:

```

```

:
:
:
<div class="article">
  
  <p>这是一段测试文字</p>
</div>
:
:
:

```

作用2: 实现块级标签排在一行的页面布局

例如

```

:
:
:
.item {
  float: left;
  width: 100px;
  height: 40px;
  background-color: grey;
  margin-right: 5px;
}
:
:
:

```

```

:
:
:
<div class="item">1</div>
<div class="item">2</div>
<div class="item">3</div>
:
:
:

```

浮动的特点:

- ① 浮动元素会脱离标准流, 在标准流中不占位置
- ② 浮动找浮动, 下一个浮动会在上一下浮动元素后面左右浮动
- ③ 浮动有特殊的显示效果, 一行显示多个并且可以设置宽高
- ④ 浮动后的元素不能通过 `text-align: center;` 或 `margin: 0 auto;` 来设置

清除 float 浮动影响的两种方法？

① 给父元素加高度

② 给父元素设置 overflow: hidden;

产生原因：

当父元素的子元素使用了 float 且父元素没有被设置固定高度时，

由于子元素脱离了正常文档流，“浮动”起来了，本来应被子元素撑开的父元素就“高度塌陷”了

overflow 属性 定义溢出元素内容区的内容如何处理

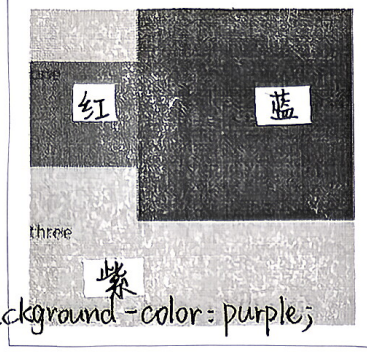
值	描述
Visible	默认值，内容不会被修剪而是呈现在元素框之外
hidden	内容会被修剪且其余内容不可见
scroll	内容会被修剪且浏览器强制显示滚动条，可查看其余内容
auto	若内容被修剪，则浏览器显示滚动条以便查看其余内容
inherit	从父元素继承 overflow 属性的值

例. <!DOCTYPE html>

```
<html lang="zh-CN">
<head>
<meta charset="UTF-8">
<title>浮动案例1</title>
<style>
```

```
.container{
width: 150px; height: 150px; background-color: purple;
}
.blue{
float: right;
width: 100px; height: 100px; background-color: blue;
}
.purple{
float: left;
width: 50px; height: 25px; background-color: purple;
}
.red{
float: left;
width: 50px; height: 50px; background-color: red;
}
.purple2{
float: left;
width: 50px; height: 75px; background-color: purple;
}
```

例. </style>
</head>

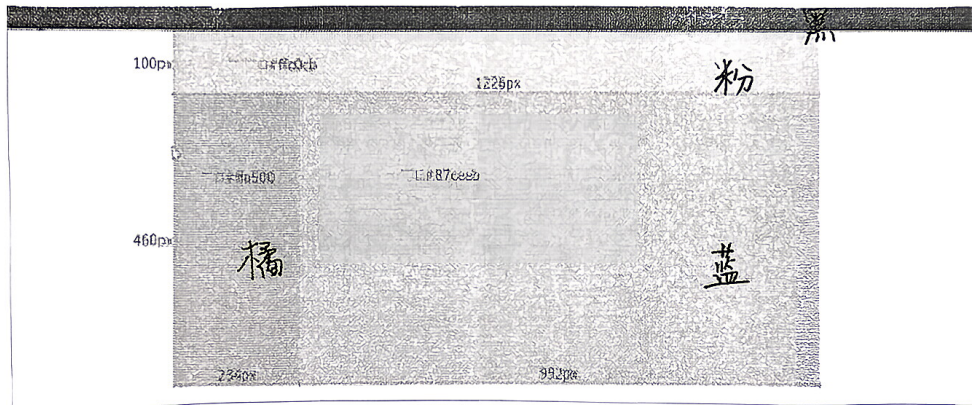


<body>

```
<div class="container">
<div class="blue">two</div>
<div class="purple"></div>
<div class="red">one</div>
<div class="purple2">three</div>
</div>
```

</body>

</html>



<!DOCTYPE html>
<html lang="zh-CN">
<head>

```
<meta charset="UTF-8">
<title>浮动案例2</title>
<style>
*{ margin: 0; padding: 0; box-sizing: border-box; }
.black{
width: 100%;
height: 40px;
background-color: black;
}
.content{
width: 1226px;
margin: 0 auto;
}
.pink{
height: 100px;
background-color: pink;
}
.orange{
float: left;
width: 234px; height: 460px; background-color: orange;
}
.blue{
float: right;
width: 992px; height: 460px; background-color: blue;
}
</style></head>
```

<body>

```
<div class="black"></div>
```

```
<div class="content">
```

```
<div class="pink"></div>
```

```
<div class="orange"></div>
```

```
<div class="blue"></div>
```

```
</div>
```

</body>

</html>